This document is a part of the CAPE-Twiggs HSSSI-26 SlimSat Project Documentation

PREPARED BY:

	Eric Tapio
	HSSSI-26 SFBA Space Penguins Space Mission Mentor
	etapio@alumni.stanford.edu

Record of Revisions

REV	DATE	AFFECTED PAGES	DESCRIPTION OF CHANGE	AUTHOR(S)
A Draft	2/18/26	All	Initial Draft	Eric Tapio

Table of Contents

Page

1 INTRODUCTION

1.1 Purpose

This document provides a structured process for HSSSI-26 student teams to follow enabling first the development and then the test of SlimSat Payloads. Specifically, this document walks through a 4-step process that assists students through the selection of candidate payload sensors (identifying sensor electrical interface needs), software development to operate the selected payload sensor, the software integration of the student payload into the SlimSat Bus, and testing and verification of proper functioning of the student payload through the SlimSat Bus command interface. The complete process is performed and documented using an example payload sensor.

1.2 Scope

Refences to resources that may be helpful for performing SlimSat payload development, including links to HW used, software required, and guides and documentation is presented in Section 2. Acronyms and definitions used throughout this document are elaborated in Section 3. Equipment required to get started with SlimSat Payload development is documented in Section 4. Prior to describing the payload development process, the overall SlimSat Concept of Operations (ConOps) is described in Section 5. Each step of the 4-Step payload development process is then described and presented in Section 6. The complete 4-Step payload development process is then applied to example sensor in Section 7, which provides a concrete example for student teams to follow.

2 REFERENCES, DOCUMENTS, & LINKS

2.1 Links

Link #	Link Description	Link Title
1	SlimSat SW Github Repo	https://github.com/eric-tapio/HSSSI26_SlimSat
2	Arduino IDE Download	https://www.arduino.cc/en/software/
3	Ping Sensor	https://www.parallax.com/product/ping-ultrasonic-distance-sensor/
4	ItsyBitsy nRF52840 Microcontroller	https://learn.adafruit.com/adafruit-itsybitsy-nrf52840-express
5	ItsyBitsy nRF52840 Microcontroller Pinout	https://learn.adafruit.com/adafruit-itsybitsy-nrf52840-express/pinouts
6	SlimSat Software Installation Guide	https://github.com/eric-tapio/HSSSI26_SlimSat/blob/main/CAPE_Twiggs_HSSSI-26_SlimSat_Software_Installation_Guide.pdf
7	SlimSat Software Description Document	TBD
8	Difference Between Procedural and Object-Oriented Programming	https://www.geeksforgeeks.org/software-engineering/differences-between-procedural-and-object-oriented-programming/
9	CAPE-Twiggs Con Ops Document	TBD

3 ACRONYMS AND DEFINITIONS

ADC	Analog to Digital Converter
ConOps	Concept of Operations
GS	Ground Station
GPIO	General Purpose Input/Output
H/W	Hardware
HSSSI	High School SlimSat Space Initiative
IDE	Integrated Development Environment
I2C	Inter-Integrated Circuit
I/F	Interface
ITAR	International Traffic in Arms Regulations
LoRa	Long Range (Radio)
NMEA	National Marine Electronics Association
OOP	Object-Oriented Programming
P/L	Payload
PWM	Pulse Width Modulation
S/C	Spacecraft
S/W	Software
UART	Universal Asynchronous Receiver Transmitter

4 ITEMS REQUIRED

Item #	Description	Qty	✔
1	Computer with Arduino IDE installed	1	
2	ItsyBitsy nRF52840 Microcontroller	1	
3	USB C (or USB A) to Micro USB Cable/Adapter	1	
4	Candidate Sensor(s) (Ping Sensor is used as an example)	1	
5	Solderless Breadboard	1	
6	Jumper Wires		

5 HIGH-LEVEL SLIMSAT CONCEPT OF OPERATIONS

The high-level SlimSat mission concept of operations is shown below in Figure 1.

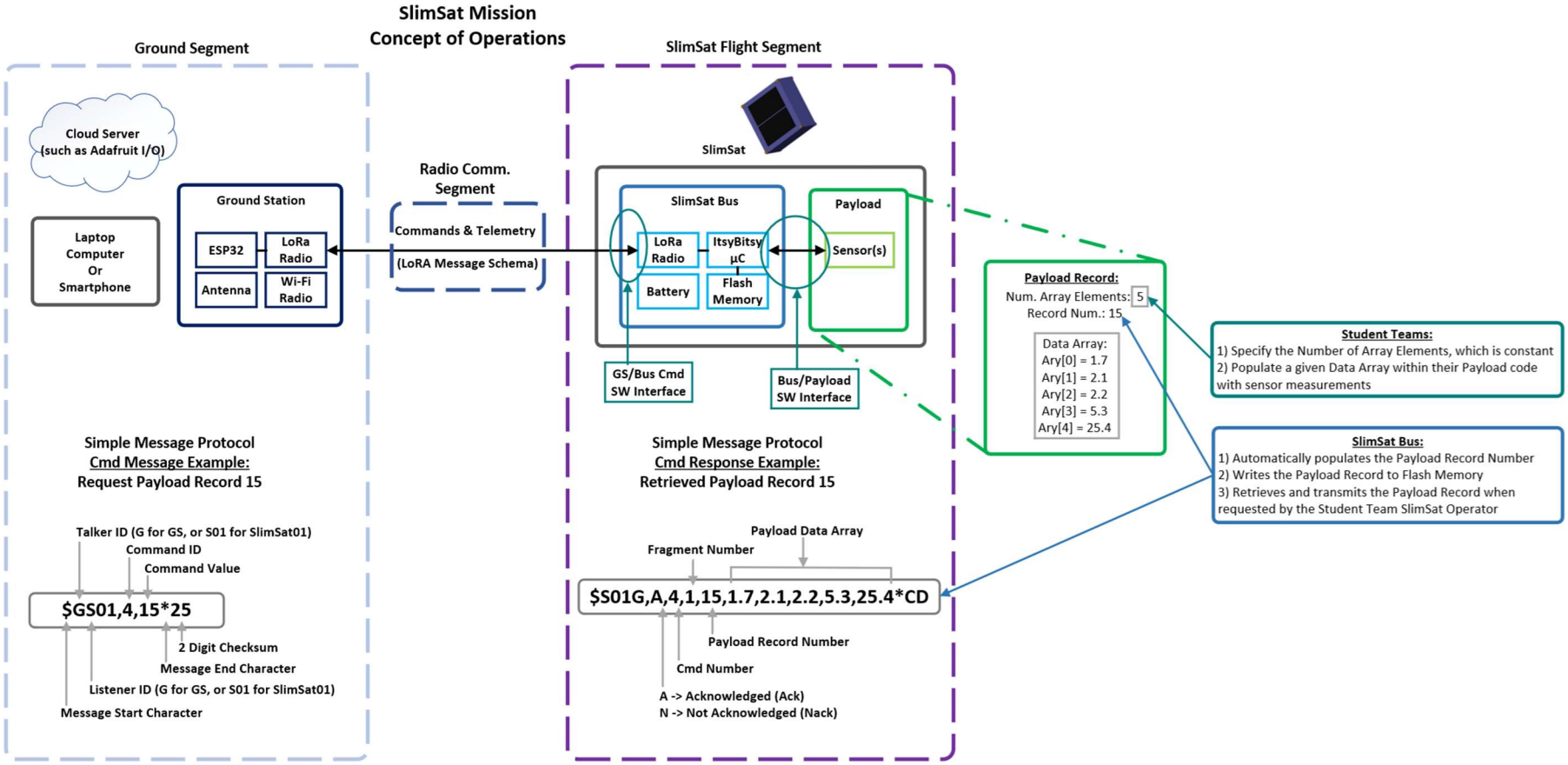

Figure 1 – High-Level HSSSI-26 SlimSat Concept of Operations

As identified in Figure 1, there are three major segments to the SlimSat ConOps: a Ground Segment, a Radio Communications Segment, and the SlimSat Flight Segment. The Ground Segment encompasses all the hardware and software elements that are required to connect an operator with the SlimSat, including ground station hardware, a computer to enter commands to send to the SlimSat, and a server to store data downlinked from the SlimSat.

The Radio Communications Segment encompasses the command and telemetry protocol used to communicate between the Ground Station and the SlimSat. This segment is currently being developed by the University of Louisiana, Lafayette Ground Station Team, and will be fully implemented at a later date. Until the new protocol is released, a simple message protocol inspired by the NMEA 0183 protocol, which is a simple ASCII protocol that defines how data are transmitted in a "sentence" from one "talker" to one or more "listeners" at a time, for SlimSat communication.

The SlimSat Flight Segment is comprised of the SlimSat Bus hardware, the Payload Hardware, the SlimSat Bus software and the payload software. During nominal operations, the SlimSat will regularly record SlimSat Bus data records, which include battery voltage, SlimSat system current, temperature and write the Bus data to flash memory, where it can be retrieved by command. The SlimSat will also regularly record Payload Data records and write the data to flash memory, where it can be retrieved by command.

During SlimSat operations, Student Team SlimSat Operators can send multiple commands to the SlimSat including retrieving SlimSat Bus Data records and payload data records. With the simple message protocol, the SlimSat stored payload data record gets formatted into a "sentence" and downlinked to the ground where Student teams can then analyze the payload data.

Now that the overall ConOps of the SlimSat has been described at a high level, the 4-step Payload development process is described starting in the next session.

6 SLIMSAT PAYLOAD DEVELOPMENT PROCESS OVERVIEW

This section describes the HSSSI-26 4-step SlimSat Payload development process. The 4-step development process is illustrated below in Figure 2.

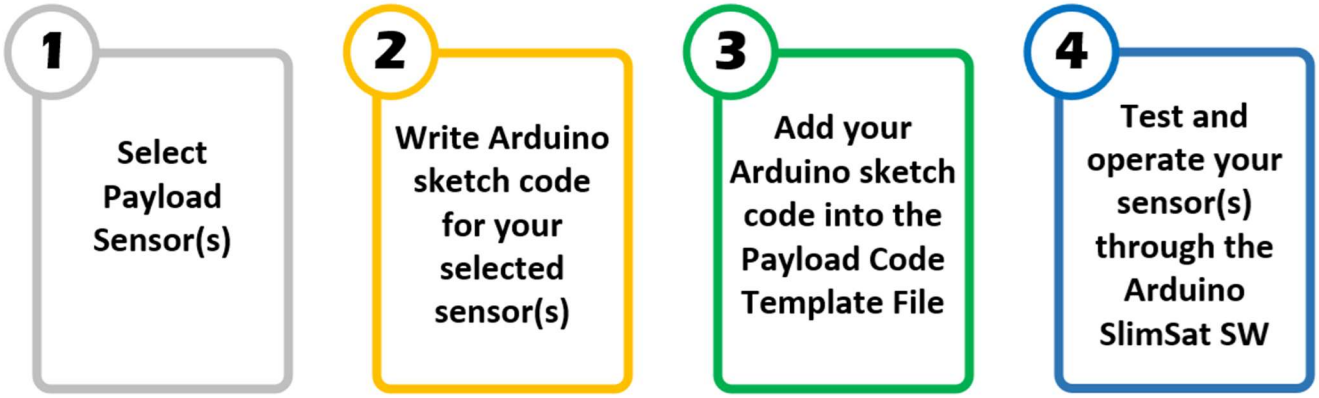

Figure 2 – HSSSI-26 SlimSat Payload Development 4-Step Process

The end objective of each step in the process and the success criteria for each step is documented in this section starting with Step 1. Additionally, the entire 4-step process is carried out, and the specific success criteria artifacts for each step are documented for an example sensor in Section 7.

6.1 STEP 1 – PAYLOAD SENSOR SELECTION

Step Objective: The objective of this step is to select sensors that both support the student team's decided SlimSat mission purpose and are compatible with the SlimSat bus constraints and interfaces, including physical size, power, data interface, data type, and data volume.

Table 1: Step 1 Success Criteria Table

Item #	Step Success Criteria	✔
1	Selected candidate sensor(s) meets the general requirements listed in this section	
2	Understands the data produced by the sensor provides and how the measured data will be used by the student team to answer of fulfil the decided SlimSat (science) mission purpose	
3	Completed a wiring diagram showing how the candidate sensor(s) connect to the SlimSat Bus and ItsyBitsy microcontroller, including power source (such as 3.3V), ground connection, and data bus connections	

HSSSI-26 student teams will select their own payload sensors to fly on their SlimSats. Students are encouraged to be creative and ingenious with their consideration of payload sensors and experimentation for their SlimSat missions.

The purpose of this section is to provide some guidance to student teams to help by identifying the important considerations for candidate sensors.

The driving constraints for small spacecraft systems are generally: physical size, power, and downlink data rate. There is a plethora of sensors available. When it comes to selecting sensors to fly on SlimSats, several important considerations are sensor:

a) Physical size
b) Electrical interface
c) Output data type & volume

6.1.1 Payload Volume - Physical Constraints

The available payload volume provided by the SlimSat is shown below highlighted in red in Figure 3.

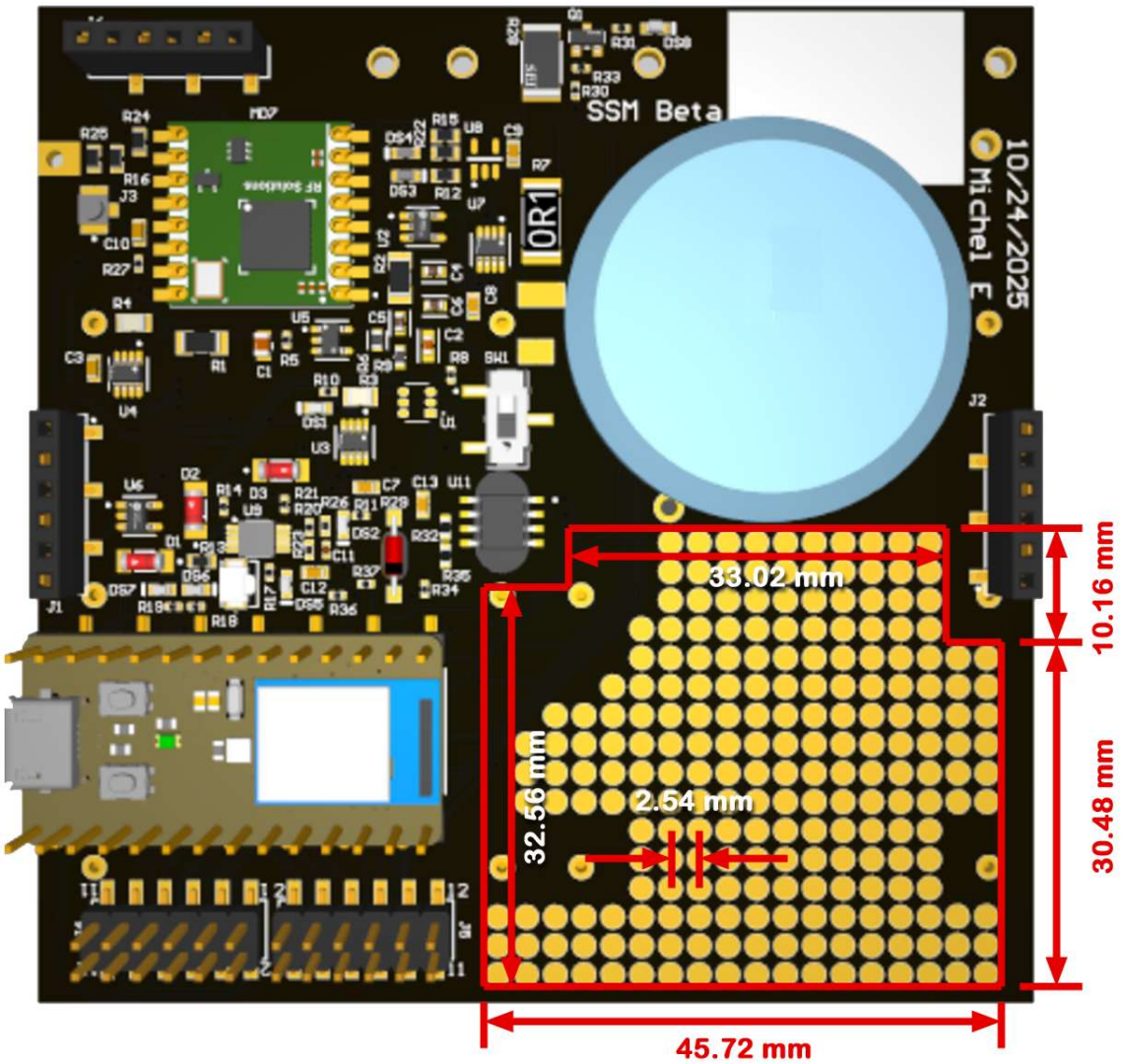

Figure 3 – SlimSat Payload Volume

The estimated payload volume is approximately 45.7 x 32.5 x 10 mm (L x W x D).

6.1.2 Electrical Interface

There is an incredibly wide variety of electrical interfaces for sensors. Additionally, there are two major classifications of electrical interfaces, both of which need to be considered when performing sensor selection: senor power electrical interface and sensor data electrical interface.

Power Interface Considerations:

The SlimSat bus provides two power interfaces: unregulated 3.6 Volts from the SlimSat battery and regulated 3.3 Volt power. Using the regulated 3.3 V power from the SlimSat will generally be the best power source to use for powering SlimSat payload sensors as it is a common power level used by many sensors.

Data Interface Considerations:

The following pins listed in Table 2 on the ItsyBitsy nRF52840 microcontroller may be used by student teams for interfacing and controlling the payload.

Table 2: ItsyBitsy nRF52840 Pins Available for Payload Interface and Control (Preliminary)

Pin Name	Pin Type / Description	Pin Bus Usage
A1, A2, A3, A4, A5	Analog Input or Digital Input or Digital Output	Analog to Digital Converter (ADC) input or Low-Speed digital input or output
D5, D7, D9, D11, D12	Digital Input or Digital Output	High-Speed Digital Input or Digital Output, including optional SPI Bus
SDA, SCL	I2C Data and Clock	Digital I2C Bus
RX, TX	UART Receive and Transmit	UART Bus

For additional information on the ItsyBitsy nRF52840 pinout, see Ref. 5.

There are several data interface types directly supported by the ItsyBitsy nRF52840 microcontroller. These include digital 3.3V logic level signals (such as General-Purpose Input/Output (GPIO) lines, and Pulse Width Modulation (PWM)), analog voltage output, analog resistance (ohms) output, I2C bus, SPI, or UART. Sensors that use any of these data interfaces are well suited for use with the SlimSat. Additional data interfaces may also be supported. If you have questions about electrical or data interfaces, we encourage you to inquire with the SlimSat team.

6.1.3 Output Data Type & Volume

One of the primary constraints is communication data rate between the SlimSat and the Ground station. Additionally, it is completely feasible that a payload could produce more data that is feasible to down-link to the ground.

The ItsyBitsy nRF52840 microcontroller includes built-in flash memory, which will be used during the SlimSat mission to store collected SlimSat bus data and payload data. This data may then be retrieved at any later time.

The Payload datatype interface between the Payload and the SlimSat bus is an array of floating-point values called a double (type). A double data type has been chosen since it can represent any numeric data including integers, float values, and Boolean values.

There is flexibility in the number of payload sensor measurements, and on the order of 10 doubles downlinked is a good rule of thumb, with 20 sensor measurements taken at once.

6.2 STEP 2 – SENSOR ARDUINO CODE DEVELOPMENT

Step Objective: The objective of this step is to create an Arduino sketch (.INO) file that interfaces the sensor(s) to the ItsyBitsy microcontroller to produce and output the sensor data.

Table 3: Step 2 Success Criteria Table

Item #	Step Success Criteria	✔
1	The sensor(s) has been successfully wired and connected to the ItsyBitsy	
2	The Arduino sketch (.INO) code runs and outputs the desired sensor data	

The second step in the SlimSat Payload development process is writing an Arduino sketch (C++ code) that interfaces and operates the student team's selected sensor(s). This step does not require any SlimSat hardware or software. Only the items identified in Section 4 are required. The Arduino IDE will be the environment used to create the Arduino sketch for this step.

To do this, connect your selected sensors to the ItsyBitsy nRF52840 microcontroller and connect the USB cable to the ItsyBitsy nRF52840 microcontroller and to your computer, as shown below in Figure 4.

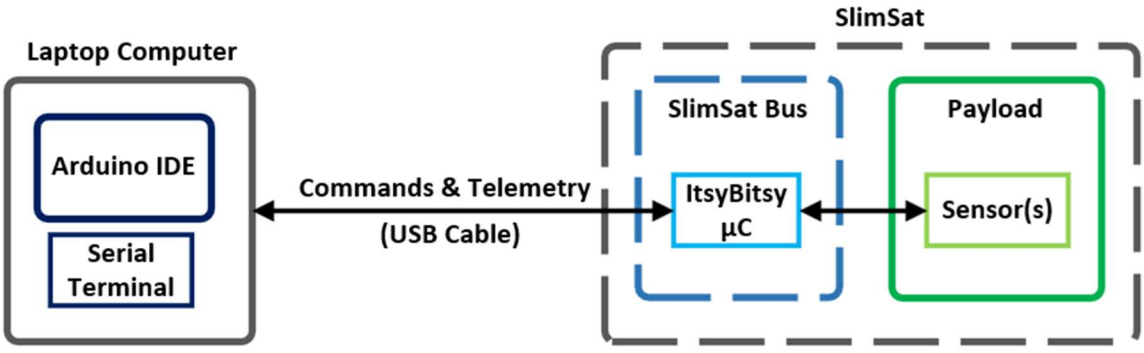

Figure 4 – High-Level Payload Development Hardware Setup

You will need to both wire and power your sensors, preferably mounted on solderless breadboard with jumper wires and any additional parts or circuits that your sensor requires, which was identified in Step 1.

A simple Arduino sketch that uses the standard Arduino Setup & Loop functions are the recommended starting point for this step of Arduino code development for the chosen payload sensors.

In this step, there is just one source code file that is needed by the end of the step, a completed Arduino sketch .INO file that interfaces with the selected sensor(s). Students may use the simple starter Arduino sketch shown below in Figure 5 as a starting point. The file can be found in its entirety in Section 8.1.

```
#include <Adafruit_TinyUSB.h> // for Serial

void setup() {
    // Put your setup code here, to run once:

    // Initialize serial communication:
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    Serial.println(" ~ Running Setup ...");

    return;
}


void loop() {
    // Put your main code here, to run repeatedly:
    Serial.println(" ~ Looping ...");

    delay(1000);

    return;
}
```

Figure 5 – Starter Arduino Sketch .INO File

When the starter Arduino sketch file shown above is uploaded and run on the ItsyBitsy, the expected output is shown below in Figure 6.

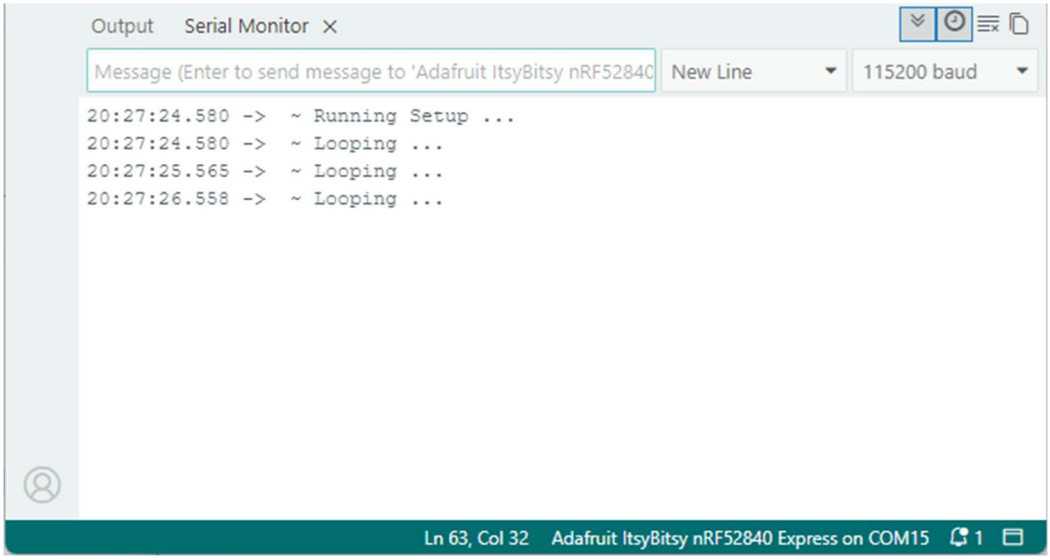

Figure 6 – Starter Arduino Sketch Output

6.3 STEP 3 – SENSOR CODE INTEGRATION INTO PAYLOAD TEMPLATE FILE

Step Objective: The objective of this step is to complete the Payload (Arduino Library) C++ code and auxiliary code files required to run and test your payload using an updated Payload Arduino sketch file.

Table 4: Step 3 Success Criteria Table

Item #	Step Success Criteria	✔
1	Complete Payload.h (which is found in the Arduino\libraries\SlimSat_Payload\src folder) with code to support your selected sensor(s)	
2	Complete Payload.cpp (which is found in the Arduino\libraries\SlimSat_Payload\src folder) with code to support your selected sensor(s)	
3	Update the PAYLOAD_DATA_ARY_SIZE value in payload_data.h (which is found in the Arduino\libraries\SlimSat_Payload_Data\src folder)	
4	Complete Run Payload.ino Arduino sketch (which is found in the Arduino\libraries\SlimSat_Payload\examples\run_payload folder) with code to support your selected sensor(s)	
5	The Arduino sketch (.INO) code runs and outputs the desired and expected sensor data	

Once student teams have developed the Arduino code to run and operate their payload sensors, the next step to largely copy and paste the developed code into a provided C++ Payload template file that is designed to work seamlessly with the developed SlimSat Bus software.

This step requires that SlimSat Software be downloaded and installed into the Arduino IDE. For the detailed instructions to download and install SlimSat software on your computer, follow the instructions outlined in Ref. 6.

Although SlimSat software is required for this step, no SlimSat hardware is required. The same setup used in Step 2 is sufficient for performing and completing this step.

The Payload Template file consists of two files, a header file, which has an extension of .h and C++ code file, which has a .cpp file extension. Although it is acceptable to have all the code in just the .cpp file, it is typical in C/C++ to partition the program into these two files. Furthermore, in the coding environment that we are using, these two files, based on their structuring, form an Arduino library.

As a result, in this step, the number of source code files grows from one file (the Arduino sketch .INO file) to three files (the Arduino sketch .INO file, the Payload Template .h file, and the Payload Template .cpp file).

The big change and challenge in this step are that a change in coding approach is made from Traditional/Procedural coding to an Object-Oriented Programming (OOP) approach.

What exactly does the change pertain?
Traditional coding (often procedural/structured) focuses on a top-down, step-by-step approach, organizing code into functions and procedures that act on separate data. In contrast, Object-Oriented Programming (OOP) organizes code into self-contained "objects" that bundle both data and behavior, following a bottom-up approach to better manage complex, real-world systems.

Why is the change needed?
The SlimSat Software is a collection of Arduino Libraries, which use many existing Arduino Libraries. Arduino Libraries are required to be OOP programs, and additionally, OOP is the contemporary coding approach for real-world systems.

For more information about the differences between Traditional/Procedural and OOP programming, see: Ref. 8.

Additional training and tutorials for student teams will be provided to help students and educators alike to better understand the structure of OOP programs and how to work with them.

The Payload Template C++ Header file is shown in Figure 7 and a text version of the file may be found in Section 8.2.

```
/**
 * @file payload.h
 * @brief SlimSat Payload System Header
 *
 * @details This header file defines the Payload class which manages the
 * SlimSat payload subsystem. It handles sensor operations, measurement
 * collection, data processing, and command execution for the payload sensors
 *
 * CAPE-Twiggs HSSSI-26 SlimSat Project
 * Copyright (c) 2025, Eric Tapio. All rights reserved.
 * Licensed under the MIT license. See LICENSE file in the project root for full license information.
 */

#ifndef SLIMSAT_PAYLOAD_HEADER
#define SLIMSAT_PAYLOAD_HEADER

#include <Arduino.h>
#include <payload_data.h>


/**
 * @brief SlimSat Payload System Class
 *
 * @details The Payload class manages the complete payload subsystem including
 * sensor operations, measurement collection, data processing, and optionally any command handling.
 * It can be expanded and customized by the Payload Team as needed for specific
 * mission requirements.
 */

class Payload {
private:
    // The private portion of the class can be changed to whatever is needed to tailor
    // the interface to support the sensor(s) used by the SlimSat Team

public:

    // ************************************************
    // The following public interface section shall not be modified.
    // Constructors
    Payload(void);

    void initializePayload(void);
    void performPayloadLoopIteration(PlDataRec& pl_data_rec);

    //
    // ************************************************
};

#endif
```

Figure 7 – Payload Template Header File

The Payload Template C++ code file is shown in Figure 8 and a text version of the file may be found in Section 8.2.

```
#include <payload.h>


/**
 * @brief Default constructor for the payload class
 * @details Initializes the payload system by setting up data members
 * and performing system initialization. This is the only constructor
 * available for the payload class.
 */
Payload::Payload(void) {
    // The default constructor, and only constructor, for the payload class

}


/**
 * @brief Initialize payload system
 * @details Performs complete payload initialization including hardware
 * configuration, pin setup, and system state initialization. Sets
 * payload to STOPPED state for safe startup.
 */
void Payload::initializePayload(void) {
    // This method initializes the payload, which can include initializing data members and hardware pin states
    // This method is intended to be called only once during setup

    return;
}


void Payload::performPayloadLoopIteration(PlDataRec& pl_data) {
    // This method performs the operations desired each time a payload operation is performed

    return;
}
```

Figure 8 – Payload Template C++ Code File

The SlimSat Payload code, like the SlimSat Bus, is an Arduino library, and hence also OOP code. The next step is to largely cut and paste the developed Arduino sketch code from Step 2 into the provided Payload Template file and make the necessary update to the PAYLOAD_DATA_ARY_SIZE value in payload_data.h based on the number of measurements that the student team's payload will produce each time the payload is run.

For the specific additions, changes, and modifications to the Payload code that is required in this step, see Section 7.3, which performs this step for an example sensor.

6.4 STEP 4 – PAYLOAD TEST & VERIFICATION USING SLIMSAT CMD INTERFACE

Step Objective: The objective of this step is to test and verify that the Payload code produces the expected data using the SlimSat Bus hardware and software.

Table 5: Step 4 Success Criteria Table

Item #	Step Success Criteria	✔
1	The Run SlimSat Arduino sketch (.INO) code (from SlimSat Bus Examples) runs and outputs the desired and expected sensor data	

Now that the payload sensor functionality has been verified with the SlimSat SW and a breadboard setup, the final step is to integrate the payload sensor(s) onto the SlimSat Bus PCB, load the completed SlimSat and Payload SW, and verify the full functionality of the Payload using the SlimSat SW command interface.

To do this, open the Run SlimSat from the Arduino IDE by clicking on: File -> Examples -> SlimSat Bus -> Run SlimSat

7 EXAMPLE SLIMSAT S/C BUS WITH PING PAYLOAD HARDWARE SETUP

This section walks through the entire 4-Step payload development process with an example sensor, the Ping Ultrasonic sensor.

7.1 STEP 1 – PING SENSOR SELECTION FOR PAYLOAD

The selected payload sensor is the Ping Ultrasonic range sensor, which has a 3-pin electrical interface: 5V, GND (0 Volt reference), and a SIG, which is a digital GPIO signal.

The Ping Ultrasonic range sensor produces range measurement of detected objects. The Ping Ultrasonic range sensor produces integer range measurements with a typical range in the range between 0 and 358 cm. The sensor produces one integer range measurement each time the sensor is activated.

Although the Ping Ultrasonic range sensor can be interfaced with the ItsyBitsy microcontroller, it is important to note here that the Ping Ultrasonic range sensor is not a suitable sensor to be flown on a SlimSat. There are a few reasons for this, which are helpful to point out. First, the Ping Ultrasonic sensor sends a signal through air and in outer space, there is no air. Hence, this sensor would not function well in space. Secondly, although this sensor has an easy digital interface and a small data volume, the power electrical interface is not compatible with the

SlimSat. The Ping Ultrasonic sensor requires regulated 5 Volt power, which is available from the ItsyBitsy nRF52840 microcontroller when powered through a USB cable. Nevertheless, the SlimSat module, which will use a 3.6 Volt battery, has no means to provide 5 Volt power for the Ping Sensor.

The wring diagram of the Ping Ultrasonic sensor with the ItsyBitsy microcontroller is shown below in Figure 9.

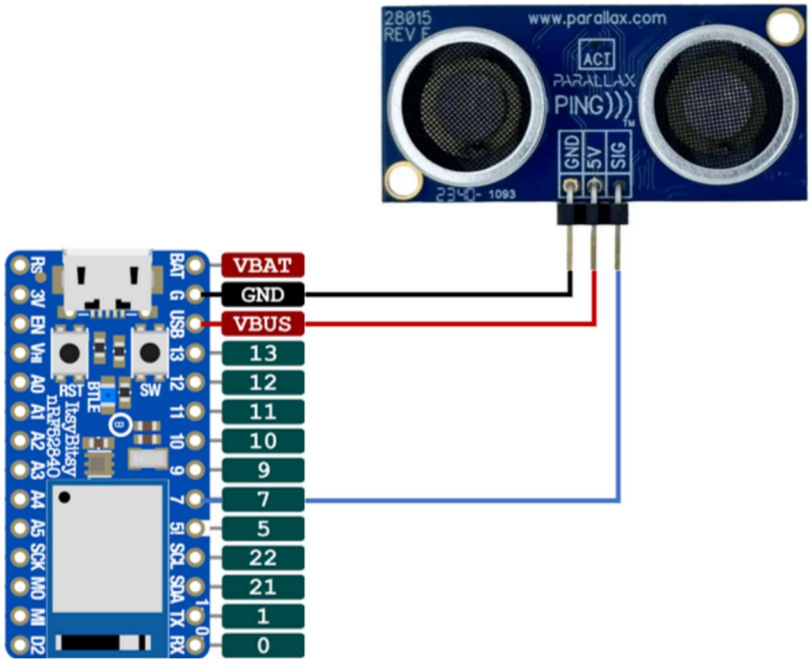

Figure 9 – Ping Sensor to Microcontroller Wiring Diagram

All the success criteria for Step 1 of payload development for the example Ping Sensor Payload has been completed, as shown below in Table 6.

Table 6: Step 1 Example Ping Sensor Payload Success Criteria Table

Item #	Step Success Criteria	✔
1	Selected candidate sensor(s) meets the general requirements listed in this section	✔
2	Understands the data produced by the sensor provides and how the measured data will be used by the student team to answer of fulfil the decided SlimSat (science) mission purpose	✔
3	Completed a wiring diagram showing how the candidate sensor(s) connect to the SlimSat Bus and ItsyBitsy microcontroller, including power source (such as 3.3V), ground connection, and data bus connections	✔

7.2 STEP 2 – PING SENSOR ARDUINO CODE DEVELOPMENT

2 Write Arduino sketch code for your selected sensor(s)

The breadboard prototype of the Ping Ultrasonic sensor connected to the ItsyBitsy microcontroller on a solderless breadboard is shown below in Figure 10.

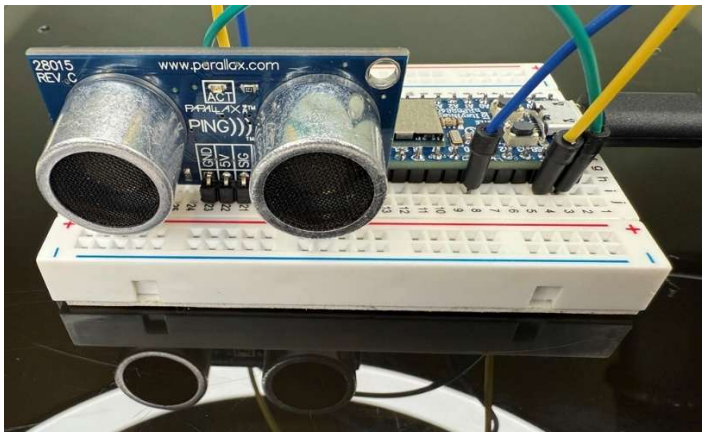

Figure 10 – Ping Sensor connected ItsyBitsy nRF52840 on Solderless Breadboard

Now that the prototype Ping Ultrasonic Payload HW has been constructed using the wiring diagram put together from Step 1, the next step is to write the Arduino code to run the Ping Sensor.

The Ping Sensor C++ Arduino code is available from the Arduino IDE Examples. Specifically, the original Ping Sensor code that was used to construct the example Ping Payload code can be found in the Arduino IDE by clicking on: File -> Examples -> 06.Sensors -> Ping

The essential code for the Ping code has been put together into an Arduino sketch .INO file, which is shown below in Figure 11. A text version of the sketch file code can be found in Section 8.3.

```
#include <Adafruit_TinyUSB.h> // for Serial
#define PING_PIN 7


long microsecondsToCentimeters(long microseconds) {
  // The speed of sound is 340 m/s or 29 microseconds per centimeter.
  // The ping travels out and back, so to find the distance of the object we
  // take half of the distance travelled.
  return microseconds / 29 / 2;
}


double getRangeMeasurementInCm(void) {
    // This method performs a Ping Sensor range measurement and returns the
  // measured range in cm

  // Establish variables for duration of the ping, and the distance result
  // in centimeters:
  long duration = 0;
  double range_in_cm = 0.0;

  // The PING))) is triggered by a HIGH pulse of 2 or more microseconds.
  // Give a short LOW pulse beforehand to ensure a clean HIGH pulse:
  pinMode(PING_PIN, OUTPUT);
  digitalWrite(PING_PIN, LOW);
  delay(3);
  digitalWrite(PING_PIN, HIGH);
  delay(5);
  digitalWrite(PING_PIN, LOW);

  // The same pin is used to read the signal from the PING))): a HIGH pulse
  // whose duration is the time (in microseconds) from the sending of the ping
  // to the reception of its echo off of an object.
  pinMode(PING_PIN, INPUT);
  duration = pulseIn(PING_PIN, HIGH);

  // Convert the time into a distance
  range_in_cm = microsecondsToCentimeters(duration);

  return range_in_cm;
}


void setup() {
  // Initialize serial communication:
  Serial.begin(115200);
  while (!Serial) {
      delay(10);
    }

  Serial.println(" ~ Running Ping Sensor Arduino Sketch ...");

  return;
}


void loop() {
  double range_in_cm = getRangeMeasurementInCm();

  // Print the range result
  Serial.print(range_in_cm);
  Serial.println(" cm");

  delay(1000);

  return;
}
```

Figure 11 – Ping Sensor Arduino Sketch .INO File

This Arduino sketch file can be run standalone and does not require any SlimSat SW. When the sketch file for the example Ping Payload is run, the output is shown below in Figure 12.

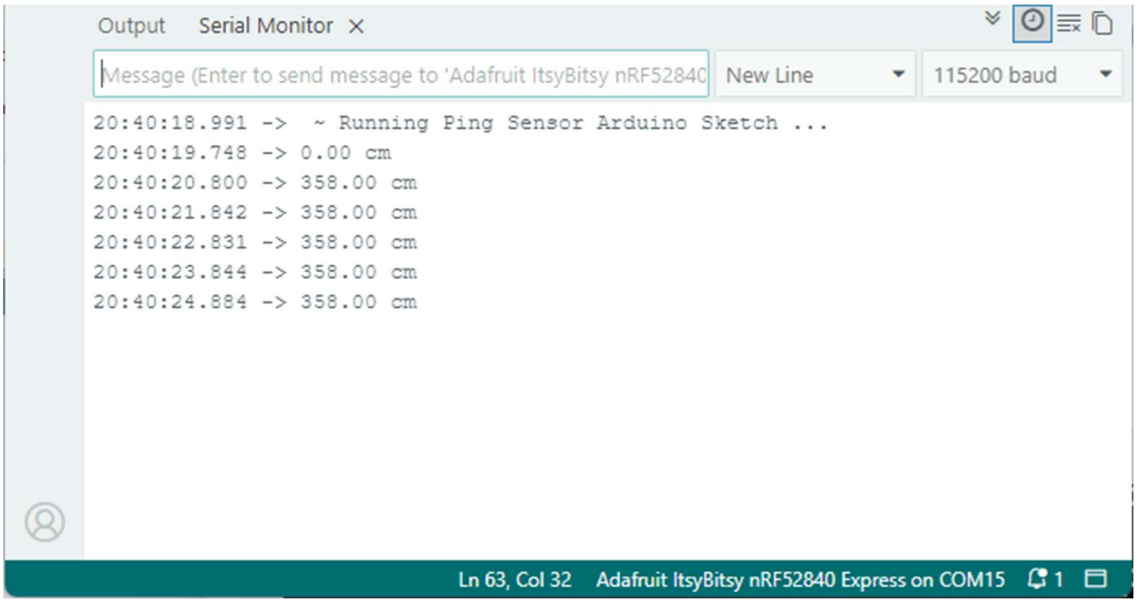

Figure 12 – Ping Sensor Arduino Sketch Output

This completes sensor-level integration, test, and verification!

All the success criteria for Step 2 of payload development for the example Ping Sensor Payload has been completed, as shown below in Table 7.

Table 7: Step 2 Example Ping Sensor Payload Success Criteria Table

Item #	Step Success Criteria	✔
1	The sensor(s) has been successfully wired and connected to the ItsyBitsy	✔
2	The Arduino sketch (.INO) code runs and outputs the desired sensor data	✔

7.3 STEP 3 – PING SENSOR CODE INTEGRATION INTO PAYLOAD TEMPLATE FILE

The next step is to largely cut and paste the developed Arduino code into the provided Payload Template file and make the necessary update to the PAYLOAD_DATA_ARY_SIZE value in payload_data.h based on the number of measurements that your payload will produce each time the payload is run.

The integrated Payload Header file code, for the example Ping Sensor in this step, is shown below in Figure 13. The specific changes that were made to the empty Payload Template Header file are annotated in the figure below.

```
/**
 * @file simple_payload.h
 * @brief SlimSat Payload System Header
 *
 * @details This header file defines the Payload class which manages the
 * payload subsystem for the SlimSat spacecraft. It handles sensor operations,
 * measurement collection, data processing, and command execution for the
 * payload instruments including ultrasonic range sensors.
 *
 * CAPE-Twiggs HSSSI-26 SlimSat Project
 * Copyright (c) 2025, Eric Tapio. All rights reserved.
 * Licensed under the MIT license. See LICENSE file in the project root for full license information.
 */

#ifndef SIMPLE_PING_PAYLOAD_HEADER
#define SIMPLE_PING_PAYLOAD_HEADER

#include <Arduino.h>
#include <payload_data.h>

#define VERBOSE_PAYLOAD_OUTPUT 0

#define PING_PIN 7

/**
 * @brief SlimSat Payload System Class
 *
 * @details The Payload class manages the complete payload subsystem including
 * sensor operations, measurement collection, data processing, and optionally any command handling.
 * It can be expanded
 * mission requirement
 */

class Payload {
private:
    // The private portion of the class can be changed to whatever is needed to tailor
    // the interface to support the sensor(s) used by the SlimSat Team

    long microsecondsToCentimeters(long microseconds);
    double getRangeMeasurementInCm(void);

public:

    // ************************************************
    // The following public interface section shall not be modified.
    // Constructors
    Payload(void);

    void initializePayload(void);
    void performPayloadLoopIteration(PlDataRec& pl_data_rec);

    //
    // ************************************************
};

#endif
```

Any #include, #define declarations, and/or global constants defined in the Arduino Sketch from Step 2 are to be added here at the top of the Payload Header file

The prototype of any functions defined in the Arduino Sketch from Step 2 are to be added here in the private section. Any global scope variables used may also be added as class data members

Figure 13 – Ping Payload Template File Integrated Header

The integrated Payload C++ code file for the example Ping Sensor in this step are shown below in Figure 14 and Figure 15. The specific changes that were made to the empty Payload Template C++ code file are annotated in the figures below.

```
#include <simple_ping_payload.h>

/**
 * @brief Default constructor for the payload class
 * @details Initializes the payload system by setting up data members
 * and performing system initialization. This is the only constructor
 * available for the payload class.
 */
Payload::Payload(void) {
    // The default constructor, and only constructor, for the payload class
    // No data members in this example
}

/**
 * @brief Initialize payload system
 * @details Performs complete payload initialization including hardware
 * configuration, pin setup, and system state initialization. Sets
 * payload to STOPPED state for safe startup.
 */
void Payload::initializePayload(void) {
    // This method initializes the payload, which can include initializing data members
    // and hardware pin states. This method is intended to be called only once during setup

    // Noth

    return;
}

void Payload::performPayloadLoopIteration(PlDataRec& pl_data) {
    // This method performs the operations desired each time a payload operation is performed

    double measurement = 0.0;

    for (uint8_t i=0; i<PAYLOAD_DATA_ARY_SIZE; i++) {
        measurement = getRangeMeasurementInCm();
        pl_data.setArrayElement(i, measurement);
    }

    return;
}
```

Add any data members defined & initialize them in the constructor

The Payload Data Record, which contains the payload array to be populated is passed into the performPayloadLoopOperation method

Use the setArrayElement method to populate payload data array elements

Figure 14 – Ping Payload Template File Integrated C++ Code: Part 1

In OOP, all objects have constructors, which construct the object and initialize data members. The Payload class has a default constructor. In general, a default constructor should be the only constructor required for the Payload class. In the example Simple Ping Sensor Payload, there are no data members that are required to be initialized, though data members may be added as necessary.

```
double Payload::getRangeMeasurementInCm(void) {
    // This method performs a Ping sensor range measurement and returns the measured range in cm

    long duration = 0;
    double range_in_cm = 0.0;

    // The PING))) is triggered by a HIGH pulse of 2 or more microseconds.
    // Give a short LOW pulse beforehand to ensure a clean HIGH pulse:
    pinMode(PING_PIN, OUTPUT);
    digitalWrite(PING_PIN, LOW);
    delay(3);
    digitalWrite(PING_PIN, HIGH);
    delay(5);
    digitalWrite(PING_PIN, LOW);

    // The same pin is used to read the signal from the PING))): a HIGH
    // whose duration is the time (in microseconds) from the sending of
    // to the reception of its echo off of an object.

    pinMode(PING_PIN, INPUT);
    duration = pulseIn(PING_PIN, HIGH);

    // Convert the time into a distance
    range_in_cm = microsecondsToCentimeters(duration);

    return range_in_cm;
}

long Payload::microsecondsToCentimeters(long microseconds) {
  // The speed of sound is 340 m/s or 29 microseconds per centimeter.
  // The ping travels out and back, so to find the distance of the object we
  // take half of the distance travelled.
    return microseconds / 29 / 2;
}
```

Add the code for any functions declared in the Arduino sketch from Step 2 into the Payload.cpp file & add "Payload::" in the prototype

Figure 15 – Ping Payload Template File Integrated C++ Code: Part 2

The updated Payload Data Header file, for the example Ping Sensor in this step, is shown below in Figure 16. The specific change that was made to the file is annotated in the figure below.

```
//
//  Payload Data Record
//
// CAPE-Twiggs HSSSI-26 SlimSat Project
// Copyright (c) 2025, Eric Tapio. All rights reserved.
// Licensed under the MIT license. See LICENSE file in the project root for full license information.

#ifndef SLIMSAT_PAYLOAD_DATA_HEADER
#define SLIMSAT_PAYLOAD_DATA_HEADER

#include <Arduino.h>

#define PAYLOAD_DATA_ARY_SIZE 10 // Change the size of the payload data arrary to suit the needs for your payload
#define PAYLOAD_DATA_VERBOSE_OUTPUT 0


class PlDataRec {
private:
    double pl_data_array[PAYLOAD_DATA_ARY_SIZE];
    double* pl_data_ptr;

public:
    uint16_t pl_rec_number;
    uint32_t time;
    PlDataRec(void);
    void initializeArray(void);
    void printRecord(void);
    void printArray(void);
    uint8_t getRecordSize(void);
    void setArrayElement(uint8_t index, double value);
    double getArrayElement(uint8_t index);
    uint32_t getTime(void);
    void setTime(uint32_t time_val);
    uint16_t getRecordNumber(void);
    void setRecordNumber(uint16_t rec_number);
};

#endif
```

The Payload Data Array Size has been set to 10 elements for the example Ping Sensor Payload

Figure 16 – Payload Data Header File with Payload Data Array Size Updated

The Payload Arduino sketch file, for the example Ping Sensor in this step, is shown below in Figure 17. Some of the important changes that were made to the starter Arduino sketch file are annotated in the figure below.

Figure 17 – Ping Payload Arduino Sketch .INO File

When the Payload Arduino sketch file for the example Ping Payload is run, the output is shown below in Figure 18.

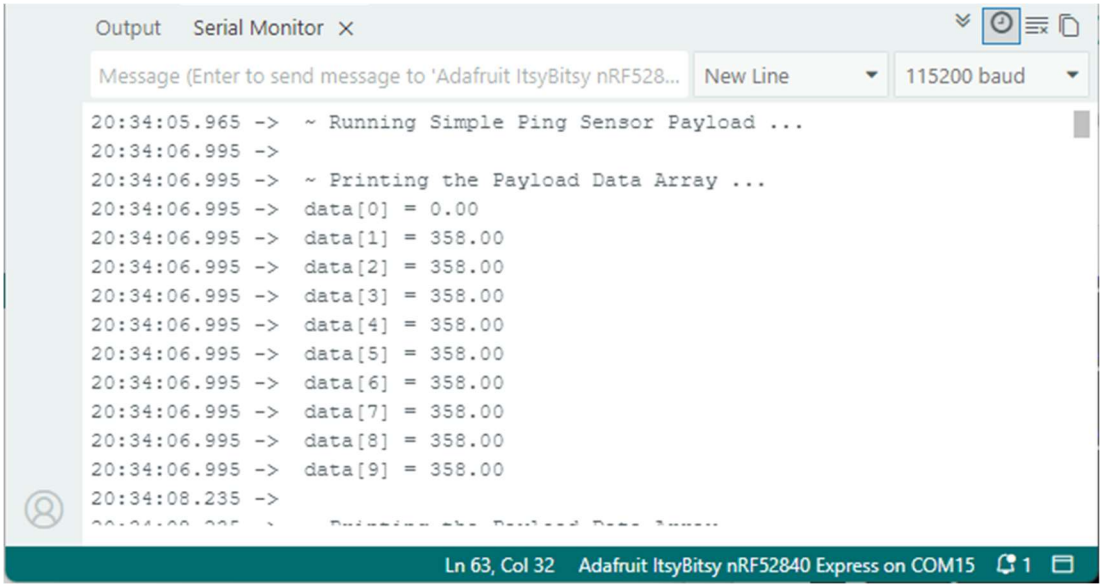

Figure 18 – Example Ping Sensor Payload Ping Payload Arduino Sketch Output

This completes payload-level integration, test, and verification!

All the success criteria for Step 3 of payload development for the example Ping Sensor Payload has been completed, as shown below in Table 8.

Table 8: Step 3 Example Ping Sensor Payload Success Criteria Table

Item #	Step Success Criteria	✔
1	Complete Payload.h, (which is found in the Arduino\libraries\SlimSat_Payload\src folder) with code to support your selected sensor(s)	✔
2	Complete Payload.cpp, (which is found in the Arduino\libraries\SlimSat_Payload\src folder) with code to support your selected sensor(s)	✔
3	Update the PAYLOAD_DATA_ARY_SIZE value in payload_data.h (which is found in the Arduino\libraries\SlimSat_Payload_Data\src folder)	✔
4	Complete Run Payload.ino Arduino sketch, (which is found in the Arduino\libraries\SlimSat_Payload\examples\run_payload folder) with code to support your selected sensor(s)	✔
5	The Arduino sketch (.INO) code runs and outputs the desired and expected sensor data	✔

7.4 STEP 4 – PING PAYLOAD TEST & VERIFICATION USING SLIMSAT CMD INTERFACE

Now that the payload sensor functionality has been verified with the SlimSat SW and a breadboard setup, the final step is to integrate the payload sensor(s) onto the SlimSat Bus PCB, load the completed SlimSat and Payload SW, and verify the full functionality of the Payload using the SlimSat SW command interface.

To do this run the latest Run SlimSat Example code in the Arduino IDE by clicking on:
File -> Examples -> SlimSat Bus -> Run SlimSat

Using the Serial Monitor enter the command to request the current payload data from the SlimSat:
$GS01,4*0D

Then verify that the SlimSat transmits a payload data response message containing the expected payload data elements, as illustrated below in Figure 19.

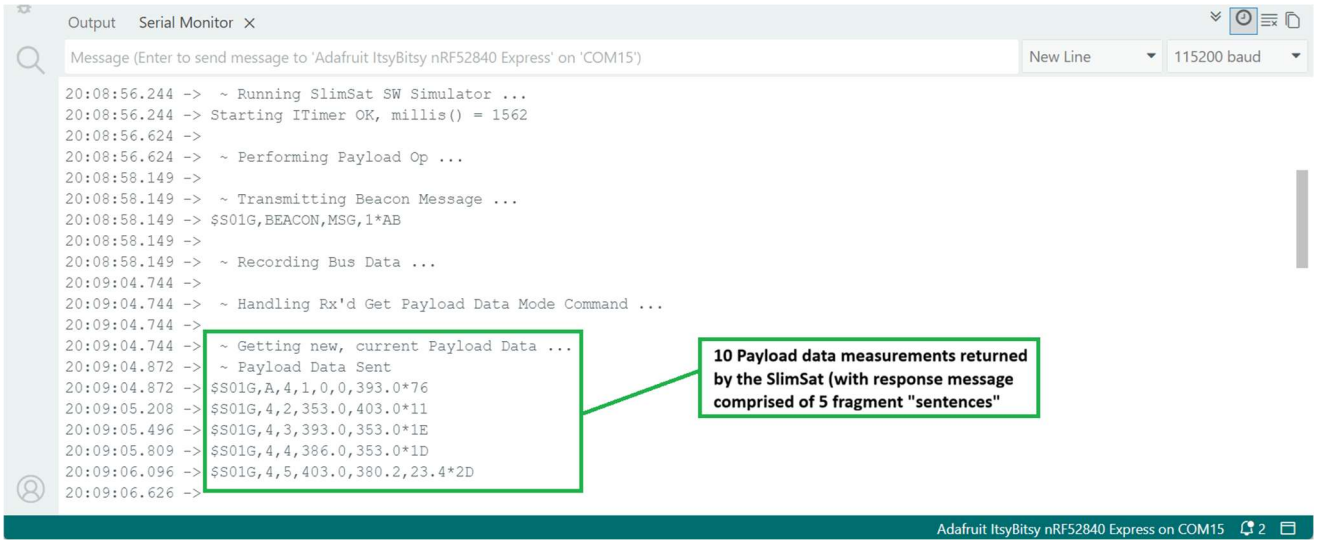

Figure 19 – SlimSat Arduino Sketch Output

Congratulations, this completes system-level SlimSat/Payload integration, test, and verification! This also completes all steps in the 4-step process for SlimSat Payload development.

All the success criteria for Step 4 of payload development for the example Ping Sensor Payload has been completed, as shown below in Table 9.

Table 9: Step 4 Example Ping Sensor Payload Success Criteria Table

Item #	Step Success Criteria	✔
1	The Run SlimSat Arduino sketch (.INO) code (from SlimSat Bus Examples) runs	✔
2	When the get current payload data command is sent to the SlimSat, the SlimSat displayed response message contains expected sensor data	✔

8 APPENDIX – SOURCE CODE

This section contains source code that is presented in this document, which may be cut and pasted into a text file or into an Arduino sketch.

8.1 Arduino Sketch Starter

A source code text version of the Arduino sketch starter is provided below.

```
#include <Adafruit_TinyUSB.h> // for Serial

void setup() {
  // Put your setup code here, to run once:

    // Initialize serial communication:
    Serial.begin(115200);
    while (!Serial) {
    delay(10);
  }

  Serial.println(" ~ Running Setup ...");

  return;
}

void loop() {
  // Put your main code here, to run repeatedly:
  Serial.println(" ~ Looping ...");

  delay(1000);

  return;
}
```

8.2 Payload Template Code Files

A source code text version of the Payload Template Header .h file is provided below.

```
#ifndef SLIMSAT_PAYLOAD_HEADER
#define SLIMSAT_PAYLOAD_HEADER

#include <Arduino.h>
#include <payload_data.h>

/**
 * @brief SlimSat Payload System Class
 *
 * @details The Payload class manages the complete payload subsystem including
```

```
 * sensor operations, measurement collection, data processing, and optionally any command handling.
 * It can be expanded and customized by the Payload Team as needed for specific
 * mission requirements.
 */

class Payload {
private:
        // The private portion of the class can be changed to whatever is needed to tailor
        // the interface to support the sensor(s) used by the SlimSat Team

public:

        // ************************************************
        // The following public interface section shall not be modified.
        // Constructors
        Payload(void);

        void initializePayload(void);
        void performPayloadLoopIteration(PlDataRec& pl_data_rec);

        //
        // ************************************************
};

#endif
```

A source code text version of the Payload Template C++ .cpp file is provided below.

```
#include <payload.h>

/**
 * @brief Default constructor for the payload class
 * @details Initializes the payload system by setting up data members
 * and performing system initialization. This is the only constructor
 * available for the payload class.
 */
Payload::Payload(void) {
        // The default constructor, and only constructor, for the payload class

}

/**
 * @brief Initialize payload system
 * @details Performs complete payload initialization including hardware
 * configuration, pin setup, and system state initialization. Sets
 * payload to STOPPED state for safe startup.
 */
void Payload::initializePayload(void) {
```

```
        // This method initializes the payload, which can include initializing data members and
hardware pin states
        // This method is intended to be called only once during setup

        return;
}

void Payload::performPayloadLoopIteration(PlDataRec& pl_data) {
        // This method performs the operations desired each time a payload operation is
performed

        return;
}
```

8.3 Example Ping Payload Code Files

A source code text version of the example Ping Payload Header .h file is provided below.

```
/**
* @details This header file defines the Payload class which manages the
 * payload subsystem for the SlimSat spacecraft. It handles sensor operations,
 * measurement collection, data processing, and command execution for the
 * payload instruments including ultrasonic range sensors.
 *
*/

#ifndef SIMPLE_PING_PAYLOAD_HEADER
#define SIMPLE_PING_PAYLOAD_HEADER

#include <Arduino.h>
#include <payload_data.h>

#define VERBOSE_PAYLOAD_OUTPUT 0
#define PING_PIN 7

/**
 * @brief SlimSat Payload System Class
 *
 * @details The Payload class manages the complete payload subsystem including
 * sensor operations, measurement collection, data processing, and optionally any command
handling.
 * It can be expanded and customized by the Payload Team as needed for specific
 * mission requirements.
 */

class Payload {
private:
        // The private portion of the class can be changed to whatever is needed to tailor
```


```
        // the interface to support the sensor(s) used by the SlimSat Team

        long microsecondsToCentimeters(long microseconds);
        double getRangeMeasurementInCm(void);

public:
        // ***************************************************
        // The following public interface section shall not be modified.
        // Constructors
        Payload(void);

        void initializePayload(void);
        void performPayloadLoopIteration(PlDataRec& pl_data_rec);

        //
        // ***************************************************
};

#endif
```

A source code text version of the example Ping Payload C++ .cpp file is provided below.

```
#include <payload.h>

/**
 * @brief Default constructor for the payload class
 * @details Initializes the payload system by setting up data members
 * and performing system initialization. This is the only constructor
 * available for the payload class.
 */
Payload::Payload(void) {
        // The default constructor, and only constructor, for the payload class
        // No data members in this example
}

/**
 * @brief Initialize payload system
 * @details Performs complete payload initialization including hardware
 * configuration, pin setup, and system state initialization.
 */
void Payload::initializePayload(void) {
        // This method initializes the payload, which can include initializing data members
        // and hardware pin states. This method is intended to be called only once during setup

        // Nothing to Initialize in this example

        return;
```


```
}

void Payload::performPayloadLoopIteration(PlDataRec& pl_data) {
	// This method performs the operations desired each time a payload operation is
performed

	double measurement = 0.0;

	for (uint8_t i=0; i<PAYLOAD_DATA_ARY_SIZE; i++) {
		measurement = getRangeMeasurementInCm();
		pl_data.setArrayElement(i, measurement);
	}

	return;
}

double Payload::getRangeMeasurementInCm(void) {
	// This method performs a Ping Sensor range measurement and returns the measured
range in cm

	long duration = 0;
	double range_in_cm = 0.0;

	// The PING))) is triggered by a HIGH pulse of 2 or more microseconds.
	// Give a short LOW pulse beforehand to ensure a clean HIGH pulse:
	pinMode(PING_PIN, OUTPUT);
	digitalWrite(PING_PIN, LOW);
	delay(3);
	digitalWrite(PING_PIN, HIGH);
	delay(5);
	digitalWrite(PING_PIN, LOW);

	// The same pin is used to read the signal from the PING))): a HIGH pulse
	// whose duration is the time (in microseconds) from the sending of the ping
	// to the reception of its echo off of an object.

	pinMode(PING_PIN, INPUT);
	duration = pulseIn(PING_PIN, HIGH);

	// Convert the time into a distance
	range_in_cm = microsecondsToCentimeters(duration);

	return range_in_cm;
}

long Payload::microsecondsToCentimeters(long microseconds) {
  // The speed of sound is 340 m/s or 29 microseconds per centimeter.
  // The ping travels out and back, so to find the distance of the object we
```


```
  // take half of the distance travelled.
        return microseconds / 29 / 2;
}
```

A source code text version of the example Ping Payload Arduino sketch is provided below.

```
#include <Adafruit_TinyUSB.h> // for Serial
#include <payload.h>

// Construct the Payload object
Payload Ping_Payload;

// Construct the Payload Data object
PlDataRec pl_data;

void setup() {
  // Put your setup code here, to run once:

  // Open a serial port for SlimSat communication
  Serial.begin(115200);
  while (!Serial) {
    // Wait for Serial
    delay(10);
  }

  Serial.println(" ~ Running Simple Ping Sensor Payload ...");

  // Initialize the payload
  Ping_Payload.initializePayload();

  return;
}

void loop() {
  // Test The Ping Payload Interface
  Ping_Payload.performPayloadLoopIteration(pl_data);

  // Print the range results
  pl_data.printArray();

  delay(1000);
}
```